

Hyper-Local Severe Weather Nowcasting Web App

Phased Development, Architecture, AI Execution Protocol, Review Loop & Deployment Plan

1. Project North Star

Build a fully deployable web application that continuously converts authorized satellite, atmospheric and terrain observations into hyper-local severe-weather risk forecasts for thunderstorms, cloudbursts and flash floods, with a 2–6 hour actionable window, a GIS dashboard, explainable outputs and categorized alerts.

The product should feel like a serious decision-support console rather than a consumer weather app: calm, precise, map-first, responsive, accessible and transparent about data freshness and uncertainty.

2. Non-Negotiable Engineering Rules

- One phase at a time. The AI must stop after each phase and review the whole repository before the next phase.
- Supabase is the authoritative application backend: Postgres/PostGIS, Auth, Storage, Realtime and Edge Functions where appropriate.
- Heavy scientific/ML computation belongs in a separately deployable Python inference service. Supabase remains the system-of-record and orchestration layer.
- No secrets in frontend code, Git, screenshots or documentation examples.
- No invented access to INSAT/MOSDAC, IMDAA, DEM, SMS, email or other external services.
- No fabricated accuracy claims. Every metric must identify dataset, split, horizon, geography and evaluation method.
- Synthetic/replay data is allowed for development and demos, but must be explicitly labeled.
- All forecasts display data timestamp, forecast generation timestamp, model version and freshness state.
- The UI must support loading, empty, stale, error and degraded-service states.
- Every major feature needs a test and a clear Definition of Done.

3. Recommended Technology Stack

Layer	Technology	Why
Frontend	Next.js + TypeScript	Strong routing, server/client component model, production deployment ecosystem and good fit for a responsive data-heavy web application.
UI	Tailwind CSS + shadcn/ui	Fast construction of consistent accessible components while keeping the visual system subtle and controllable.
Maps	MapLibre GL JS	Interactive vector maps without locking the project to one proprietary map SDK. Tile provider remains a separate deployment/configuration choice.
Charts	Recharts / SVG	Simple time-series and probability charts that fit the dashboard without introducing a heavy visualization framework.
Backend	Supabase	Postgres database, Auth, Storage, Realtime and Edge Functions in one managed backend platform.
Database	PostgreSQL + PostGIS	Relational integrity plus geospatial indexing/querying for grids, regions, observations and risk zones.
Lightweight server logic	Supabase Edge Functions	Secure orchestration, alert dispatch, webhooks and API mediation. Not intended for heavy ML inference.
ML service	Python + FastAPI	Suitable for Xarray/Pandas/PyTorch scientific workflows and long-running model inference in a container.
Scientific data	Xarray + Pandas + NumPy	Natural representation of multidimensional atmospheric grids and tabular metadata.
Geospatial processing	GeoPandas + Rasterio	Practical processing for vector boundaries and DEM/raster data.

Layer	Technology	Why
Deep learning	PyTorch	Flexible for spatiotemporal multi-task models and reproducible training/inference.
Testing	Vitest/Jest + Playwright + Python pytest	Unit, API and browser-level end-to-end testing.
Deployment	Vercel + Supabase + container platform	Simple separation of concerns: web frontend, managed backend and ML runtime.

4. Target Architecture

Browser → Next.js application → Supabase Auth/API → Supabase Postgres/PostGIS/Storage/Realtime → Supabase Edge Functions for lightweight orchestration → Python FastAPI inference service → scientific preprocessing/model → forecast results back into Supabase → GIS dashboard + alert workflow.

Keep the ML service stateless where practical. Persist model metadata and forecast results in Supabase. Use object storage for larger model/data artifacts when appropriate. Treat the model service as replaceable: the application should not depend on a specific model implementation.

5. Phase-by-Phase Execution Plan

Phase 0: Project Constitution & Guardrails

Freeze the scope, architecture, data contracts, UI principles, folder structure, environment strategy, security rules, and acceptance criteria before writing application code. Create the project constitution that every later phase must follow.

Implementation checklist:

- Web app only; no native mobile app in the MVP.
- Primary backend platform: Supabase for Postgres, Auth, Storage, Realtime and Edge Functions.
- Frontend: Next.js + TypeScript.
- UI: Tailwind CSS + shadcn/ui, with restrained motion and accessible states.
- Maps: MapLibre GL JS with a compatible tile provider.
- Charts: Recharts or lightweight SVG-based charts.
- Heavy ML inference: Python service/container, called through a controlled API. Do not force heavy model inference into Edge Functions.
- Data/ML: Python, Pandas, Xarray, NumPy, Scikit-learn/PyTorch as justified by the model.
- Geospatial: PostGIS, GeoPandas, Rasterio where needed.
- Deployment target: Vercel for Next.js; Supabase for backend; a container platform for Python inference.
- Never invent live INSAT/MOSDAC/IMDAA credentials, APIs, endpoints, data schemas or permissions.

Definition of Done:

- Repository initializes and builds.
- Environment variables are documented with .env.example.
- Architecture diagram and data contracts exist.
- No secrets committed.
- Definition of Done and phase-review protocol are written.

Phase 1: UX Foundation & Storytelling Shell

Build the visual identity and application shell without pretending that live weather predictions exist. Establish the serious weather/disaster-management tone: calm, precise, readable, map-first, and information-dense without being cluttered.

Implementation checklist:

- Create responsive desktop/tablet/mobile layouts.
- Build header, navigation, map canvas, risk summary panel, alert drawer, timeline, data freshness indicator and system status.
- Use restrained typography, neutral surfaces, clear severity hierarchy, subtle weather-inspired visual language.
- Use motion only for meaningful state changes: loading, risk transitions, alert arrival, map layer changes.
- Build reusable components and accessible keyboard/focus states.
- Create empty/loading/error/no-data states for every data-driven component.

Definition of Done:

- All major routes render.
- No placeholder lorem ipsum.

- No hard-coded fake live alerts presented as real data.
- Responsive behavior verified at common breakpoints.
- UI remains understandable without color alone.

Phase 2: Supabase Backend Foundation

Turn Supabase into the authoritative application backend. Establish database schema, Row Level Security, roles, auditability, storage and safe server-side access.

Implementation checklist:

- Create tables for profiles/roles, locations/grid cells, observations, forecasts, risk maps, alerts, model runs, data-ingestion status and audit events.
- Enable PostGIS if required by the selected Supabase environment.
- Define spatial indexes and time indexes.
- Implement RLS policies by role.
- Use Supabase Auth for authenticated authority users.
- Use Edge Functions for lightweight orchestration, alert routing and secure server-side operations.
- Do not expose service-role keys to the browser.
- Create migrations and seed scripts.

Definition of Done:

- Migrations run from a clean database.
- RLS tests cover authorized and unauthorized access.
- Frontend can authenticate and read only permitted data.
- Audit records are generated for sensitive actions.
- Schema/data dictionary is documented.

Phase 3: Data Ingestion & Normalization Layer

Build the data pipeline that can ingest authorized atmospheric, satellite and terrain data and convert them into a common spatiotemporal representation.

Implementation checklist:

- Define adapters for INSAT/MOSDAC, IMDAA/reanalysis and DEM sources.
- Implement source metadata, timestamps, spatial bounds, units and provenance.
- Normalize coordinate reference systems, time zones and variable names.
- Handle missing values, duplicate timestamps, malformed files and partial coverage.
- Store raw source metadata separately from processed features.
- Use deterministic preprocessing so the same input produces the same feature output.
- Start with offline/sample data before attempting live feeds.
- Create a data freshness/status service.

Definition of Done:

- A known sample dataset can be ingested end-to-end.
- Units and coordinate systems are explicit.
- Every processed record has source and timestamp metadata.
- Bad input fails safely and is observable.
- Live connectors are only enabled after human verification of access and terms.

Phase 4: Baseline Forecast Engine

Create a trustworthy baseline before deep learning. The baseline establishes measurable performance and prevents the team from claiming that a complex neural model is useful without comparison.

Implementation checklist:

- Define prediction targets for severe thunderstorm, cloudburst and flash-flood risk.
- Define forecast horizons, spatial grid, labels and evaluation windows.
- Build simple baselines such as persistence, climatology and tree-based models where appropriate.
- Create metrics appropriate to rare-event prediction: precision, recall, F1, PR-AUC, calibration, lead time and spatial hit rate.
- Create train/validation/test splits by time to avoid leakage.
- Persist model version and evaluation metadata.

Definition of Done:

- A reproducible baseline can make predictions on held-out data.
- Metrics are generated automatically.
- No temporal leakage is present.
- The dashboard can display a test forecast with an explicit 'historical/test' label.

Phase 5: Spatiotemporal Multi-Task AI Model

Implement the core deep-learning engine. A shared spatiotemporal backbone produces separate prediction heads for thunderstorm, cloudburst and flash-flood risk.

Implementation checklist:

- Start with a compact architecture such as ConvLSTM/3D CNN or another justified spatiotemporal model.
- Feed synchronized sequences of atmospheric and observational features.
- Use separate heads for the three event probabilities.
- Apply class-imbalance handling and calibrated probabilities.
- Compare against the baseline; keep the complex model only if it improves relevant metrics.
- Track model version, training data version, feature version and configuration.
- Create reproducible inference code independent from training code.

Definition of Done:

- Model trains reproducibly.
- Held-out evaluation is available.
- Three outputs are generated.
- Inference latency is measured.
- Model does not silently substitute synthetic results for live data.
- Model artifacts are versioned and the exact feature schema is recorded.

Phase 6: Terrain-Aware Flash-Flood Risk Layer

Convert severe-rainfall probability into a terrain-aware flash-flood risk estimate using DEM-derived features. This layer must be described as risk estimation, not deterministic hydrological truth unless validated.

Implementation checklist:

- Derive elevation, slope and flow-related terrain features as appropriate.
- Overlay precipitation/cloudburst probability with terrain vulnerability.
- Create a transparent risk formulation or learned model with documented assumptions.
- Separate meteorological probability from terrain susceptibility.
- Expose both components in the UI so users can understand why flood risk is elevated.

Definition of Done:

- A test region produces reproducible flood-risk maps.
- Terrain and weather layers align spatially.

- The UI distinguishes predicted weather from derived vulnerability.
- Limitations and validation status are visible.

Phase 7: Real-Time Inference & Forecast Orchestration

Connect incoming observations to the inference engine so the application can continuously update forecasts while clearly reporting data freshness and system health.

Implementation checklist:

- Implement scheduled ingestion jobs.
- Trigger preprocessing and inference only when required inputs are available.
- Use job IDs and idempotency to avoid duplicate processing.
- Persist forecast snapshots and model metadata.
- Expose API endpoints for current risk, time series and map tiles/GeoJSON as appropriate.
- Add retries, timeouts, dead-letter/error states and observability.
- Show the last successful data timestamp and forecast generation timestamp.

Definition of Done:

- A new input can trigger a complete prediction cycle.
- Duplicate jobs do not corrupt data.
- Failures are visible in the system status.
- Forecasts are retrievable through authenticated APIs.
- The UI never implies real-time freshness when the source is stale.

Phase 8: GIS Dashboard & Explainable AI

Turn predictions into an operational dashboard. The map should be the primary storytelling surface, with drill-down, timelines and explanations.

Implementation checklist:

- Build layered maps for thunderstorm, cloudburst and flash-flood probabilities.
- Add DEM/topography layer and observation overlays.
- Add time slider/forecast timeline.
- Add region/event/risk filters.
- Provide location drill-down with probability, forecast window, freshness and contributing factors.
- Implement XAI using feature attribution or transparent feature summaries appropriate to the model.
- Use accessible legends and non-color cues.

Definition of Done:

- A user can go from national/region view to a local risk cell.
- Every displayed prediction shows timestamp and model version.
- XAI explains the prediction without claiming causal certainty.
- Map remains usable on smaller screens.

Phase 9: Alerting & Authority Workflow

Translate model outputs into categorized, auditable alerts and a workflow suitable for disaster-management users.

Implementation checklist:

- Define alert levels and thresholds through configuration, not hard-coded UI logic.
- Create alert lifecycle: generated → reviewed/issued where required → acknowledged → resolved/expired.
- Implement dashboard alerts first.
- Add email/API delivery where credentials and authorization exist.

- Add SMS only after a verified provider and consent/operational process are available.
- Prevent alert spam with cooldowns, deduplication and escalation rules.
- Log who issued, acknowledged or resolved an alert.

Definition of Done:

- An alert can be generated from a forecast event.
- Duplicate alerts are suppressed.
- Recipients/roles are enforced.
- Every alert has source, timestamp, severity, area and forecast window.
- The UI distinguishes model-generated warnings from human-confirmed official warnings.

Phase 10: Security, Reliability & Production Hardening

Harden the application as a deployable production candidate.

Implementation checklist:

- Review authentication, authorization, RLS, API validation and secret handling.
- Threat-model public endpoints and ingestion endpoints.
- Add rate limiting where appropriate.
- Validate file/data inputs.
- Add structured logging, error tracking and health checks.
- Test failure of data sources, model service and database connectivity.
- Run accessibility and performance audits.
- Create backup/restore and rollback procedures.
- Document data retention and privacy assumptions.

Definition of Done:

- No client-side secret leakage.
- Critical APIs reject unauthorized access.
- Application survives expected dependency failures gracefully.
- Performance budgets are defined and tested.
- Rollback procedure is documented.

Phase 11: Deployment, Observability & Demo Readiness

Deploy the complete system and make the demonstration deterministic, credible and easy to operate.

Implementation checklist:

- Deploy Next.js frontend.
- Deploy/configure Supabase project and migrations.
- Deploy Python inference service/container.
- Configure production environment variables through platform secret managers.
- Set up domain/HTTPS if available.
- Create health/status endpoints.
- Prepare a safe demo mode using a clearly labeled historical or synthetic replay if live data is unavailable.
- Create a one-command or documented deployment/runbook.
- Perform end-to-end smoke test from data ingestion to dashboard alert.

Definition of Done:

- Fresh production deployment works from a clean environment.
- Authentication works.

- Data ingestion/inference/dashboard/alerts work end-to-end.
- Monitoring shows system health.
- Demo can run without manually editing source code.

Phase 12: Senior Full-Stack Review Loop & Final Release

Perform repeated whole-project reviews after each completed phase until review findings become repetitive, already implemented, non-actionable, or outside the agreed scope. Then freeze the release candidate.

Implementation checklist:

- Run functional review, architecture review, security review, UX review, performance review, data/ML review and deployment review.
- For every finding classify it: blocker, high, medium, low, or optional.
- Implement only changes justified by the project constitution and acceptance criteria.
- Re-test after fixes.
- Repeat review until new findings are substantially repetitive or already covered.
- Create final architecture, API, database, model and deployment documentation.
- Tag/version the release candidate.

Definition of Done:

- No unresolved blocker/high-severity issue.
- Core user journey works end-to-end.
- All critical environment variables and external dependencies are documented.
- Known limitations are explicit.
- Deployment is reproducible.
- Final release is tagged.

6. Mandatory Senior Review Loop

After every phase, the coding AI must stop before starting the next phase and perform a senior review.

Required review sequence:

1. Inspect the entire repository, not only files changed in the phase.
2. Compare implementation against the Project Constitution, architecture, current phase requirements and previous review findings.
3. Check compile/build/type errors, tests, runtime paths, security boundaries, environment variables, database migrations, API contracts and responsive UI.
4. For ML/data phases, check leakage, reproducibility, timestamps, units, coordinate systems, data provenance and whether results are being presented as real when they are synthetic/test data.
5. Produce a review report with: PASS/FAIL, findings, severity, evidence/file locations, recommended fix, and whether the issue blocks the next phase.
6. Fix blocker/high findings before proceeding.
7. Re-run relevant tests after fixes.
8. If only low-value or repetitive findings remain, explicitly record them and proceed.
9. Repeat this review after every phase. Do not batch multiple phases into one implementation turn unless the user explicitly instructs it.

The AI must never claim that a live data connector, credential, API, model performance result, deployment or external service works unless it has actually verified it in the current environment.

7. Master Instruction for the Coding AI

You are the senior full-stack engineer responsible for this project. Execute EXACTLY ONE PHASE per user instruction. Before coding: - Read the entire project and this phase plan. - Identify the current phase from the repository state. - Do not silently skip phases. - Do not redesign the architecture without explaining why and recording the decision. - Never invent credentials, APIs, datasets, endpoints, accuracy numbers or official integrations. - Prefer the smallest production-quality implementation that satisfies the current phase. During coding: - Keep the application deployable. - Use TypeScript strictly on the frontend. - Keep secrets server-side. - Use Supabase as the authoritative application backend. - Keep heavy ML inference in the Python service rather than forcing it into Supabase Edge Functions. - Preserve accessibility, responsiveness and the subtle serious weather/disaster-management visual language. - Every data-driven UI state must have loading, empty, stale and error handling. - Clearly label synthetic, replayed, historical and test data. At the end of the phase: - Run relevant tests/build/lint/type checks. - Perform the senior whole-project review using the Review Protocol. - Fix blockers/high-severity findings. - Report what was changed, what was tested, what remains, and whether human intervention is required. - STOP. Do not start the next phase automatically.

8. Human Intervention Map

Who	Phase	What must be done
You	Phase 0	Create/confirm accounts and repository; approve final scope and project name; provide any existing team GitHub/repository details.
You	Phase 2	Create the Supabase project, save the project URL/keys in the platform's secret manager, and enable required database extensions after reviewing migrations.
You	Phase 3	Obtain/confirm authorized access to INSAT/MOSDAC and IMDAA/reanalysis data. This cannot be safely invented by the AI.
You	Phase 3	Decide the initial pilot geography and acceptable spatial/temporal resolution if the official PS does not prescribe them.
You	Phase 9	Provide approved alert-delivery provider credentials if real email/SMS/API alerts are required.
You	Phase 11	Create/connect Vercel and ML deployment accounts, configure production secrets, and approve the production domain.

Who	Phase	What must be done
You	Final	Approve any claims made in the demo, especially claims about prediction accuracy, official data, real-time capability and operational use.

9. UI/UX Direction

- Visual language: serious meteorological operations console; restrained surfaces, strong hierarchy, minimal decoration.
- Storytelling: Situation → Evidence → Forecast → Impact → Action.
- Map is the hero, but never the only information source. Every color-coded risk should have a text/shape/pattern alternative.
- Use severity sparingly. Red should mean something operationally important, not simply be a brand color.
- Motion should communicate change: a forecast update, alert arrival, layer transition or loading state.
- Responsive design should preserve the decision path on mobile: current risk → location → forecast window → explanation → alert status.
- Always show uncertainty, freshness and source provenance.

10. Data & ML Safety / Anti-Hallucination Guardrails

- Never infer an undocumented live endpoint from a dataset name. Confirm the official access mechanism first.
- Never state that a dataset is available in real time unless the connector has been verified.
- Never call a risk probability a prediction accuracy. Probability is an output; accuracy is an evaluation result.
- Never train and evaluate on overlapping future information. Time-aware splitting is mandatory.
- Never hide missing data. Propagate missing/stale state into the UI and inference pipeline.
- Never imply that flash-flood risk is a deterministic hydrological forecast unless the method has been validated.
- Never present model-generated alerts as official government warnings unless an authorized workflow explicitly converts them to official alerts.
- Record source, timestamp, unit, coordinate system, preprocessing version, feature version and model version.

11. Suggested Repository Structure

/ **apps/** **web/** # Next.js + TypeScript **services/** **inference/** # Python + FastAPI + model inference **supabase/**
migrations/ **functions/** **seed/** **packages/** **ui/** **types/** **config/** **data/** **schemas/** **samples/** # small, non-sensitive development data only **ml/** **training/** **evaluation/** **configs/** **docs/** **architecture/** **api/** **data-dictionary/** **decisions/** **.env.example**

12. Final End-to-End User Story

- Authority user signs in.
- Dashboard shows current data freshness and system health.
- Map displays current risk layers.
- User selects a region and forecast horizon.
- System shows thunderstorm, cloudburst and flash-flood probabilities.
- Timeline shows how risk evolves over the next 2–6 hours.
- User opens a high-risk cell and sees contributing atmospheric/terrain factors.
- If configured thresholds are crossed, the alert engine creates a categorized system alert.
- Authorized recipients receive the alert through configured channels.
- Every action and model output is traceable through timestamps, model version and audit records.

13. Recommended Development Order Summary

Phase 0 — Project Constitution & Guardrails

Phase 1 — UX Foundation & Storytelling Shell

Phase 2 — Supabase Backend Foundation

Phase 3 — Data Ingestion & Normalization Layer

Phase 4 — Baseline Forecast Engine

Phase 5 — Spatiotemporal Multi-Task AI Model

Phase 6 — Terrain-Aware Flash-Flood Risk Layer

Phase 7 — Real-Time Inference & Forecast Orchestration

Phase 8 — GIS Dashboard & Explainable AI

Phase 9 — Alerting & Authority Workflow

Phase 10 — Security, Reliability & Production Hardening

Phase 11 — Deployment, Observability & Demo Readiness

Phase 12 — Senior Full-Stack Review Loop & Final Release

The safest sequence is: constitution → UI shell → backend → data → baseline → deep model → terrain fusion → real-time orchestration → GIS/XAI → alerts → hardening → deployment → final review.